databricks LAB-GUIDE

(https://databricks.com)

Lab Guide: AI-Powered Data Engineering (Feb-4, 2025)

0. Important

- This is your main labguide. Please **keep it open in a separate tab**. You will need it to follow the steps below and come back to them throughout the course.
- We will work with other notebooks, catalogs and workspace settings and this guide describes how things tie together, e.g. how to run DLT notebooks as a pipeline.

Super Important

This course is designed in a way it can be run with thousands of participants on a single Databricks account sharing a number of workspaces.

We are therefore using the **USER ID** (derived from your login user email) to separate schemas and pipelines and avoid namespace clashes. Just as in your own environment, you would use your company's naming schema for resources.

To get to your user id, check your login email at the to right of the workspace. Example: odl_user_1257777@databrickslabs.com (mailto:odl_user_1257777@databrickslabs.com) means your user id is: `user_1257777`

1. Add a GitHub Repo

To get access to the lab notebooks, create a repo in your workspace

Add a Git Folder

- On the left hand side, click on `Workspace` and `Home` and then use the button at the top right and click "Create / Git Folder" to add a new git folder

  - For Git Repo URL use `https://github.com/databricks/tmm` (https://github.com/databricks/tmm)
  - Git provider and repo name will be filled automatically (repo name is `tmm`).
  - Select **Sparse Checkout Mode** (otherwise you will clone more content than necessary)
  - under Clone Pattern put `Pipelines-Workshop`
  - Click "create repo" and the resoures for this course will be cloned.
- Click on `Pipelines Workshop`. This is the folder we will be working with in this lab.

5

```
# take a note of your user id (copying from top right)
#
#     USER_ID =
#
# alternatively you can run this cell to compute your USER
import random
user =
dbutils.notebook.entry_point.getDbutils().notebook().getContext().use
rName().get()
user = ''.join(filter(str.isdigit, user))

# make it work in non lab environments, create a 6 digit ID then
if len(user) < 3:
  user=str(random.randint(100000, 999999))

print(f"user_{user}")
```

2. Delta Live Tables

Understand DLT Pipelines in SQL

- Watch your instructor explaining how to get started with DLT using the DLT SQL notebook (/$./01-DLT-Loan-pipeline-SQL).
- For more information, check out the documentation: core concepts (https://docs.databricks.com/workflows/delta-live-tables/delta-live-tables-concepts.html)

After this module, you should be able to answer the following questions:

- What is the difference between Streaming Tables (ST) and Materialized Views (MV)?
- What is the CTAS pattern?
- What do we use the medallion architecture for?

Update the provided DLT pipeline for your environment

In the DLT SQL notebook (/$./01-DLT-Loan-pipeline-SQL) check if the correct volumes are used for ingestion.

- Update the folder names and locations as described in the notebook.
  - The locations used in Auto Loader must match the volumes paths as explained in the DLT SQL notebook

Run your first Data Pipeline

1. **Watch your instructor explaining how to create a DLT pipeline first**, then follow the steps below. (Detailed documentation is available here (https://docs.databricks.com/workflows/delta-live-tables/delta-live-tables-ui.html#create-a-pipeline))
2. On your workspace, under Workflows / DLT change to "Owned by me"
3. Create a new pipeline (leave all pipeline setting **on default except the ones listed below**)

- `pipeline name:` **[use your own user_id from above as the name of the pipeline]**
- Select `Serverless` to run the pipeline with serverless compute
- Under `Source Code:` select the location of the [DLT SQL notebook], that is `YOUR_USERNAME@databrickslabs.com / tmm / Pipelines-Workshop/01-DLT-L`
- For `Destination` select **Unity Catalog**
  - Catalog: demo
  - Target Schema: `your user_id` (you will **work with your own schema** to separate your content from others)
- (In older accounts **without** serverless enabled set `Cluster mode: fixed size` and `Number Workers: 1`)
- Then click "Create"

3. Click on "Start" (top right) to run the pipeline. Note, when you start the pipeline for the first time it might take a few minutes until resources are provisioned.

Note that the lab environment is configured that you can access the folders for data ingestion via Unity Catalog. Make sure to use least privilege here in a production environment. (see the official documentation for more details (https://docs.databricks.com/en/data-governance/unity-catalog/manage-external-locations-and-credentials.html))

Pipeline Graph

You can always get to your running pipelines by clicking on "Workflows" on the left menu bar and then on "Delta Live Tables" / "Owned by me"

- Check the pipeline graph
  - Identify bronze, silver and gold tables
  - Identify all streaming tables (ST) in the SQL code (use the link under "Paths" at the right to open the notebook)
  - Identify Materialized Views and Views

New Developer Experience

- Once you defined the pipeline settings the notebook is associated with the DLT code.
- On the notebook page, you will be asked if the you want to attach the notebook to the pipeline. If you do so, you can see the pipeline graph and the event log in the notebook. The notebook is then like a mini IDE for DLT.

Pipeline Settings

- Recap DLT development vs production mode
- Understand how to use Unity Catalog
- Understand DLT with serverless compute

Explore Streaming Delta Live Tables

- Take a note of the ingested records in the bronze tables
- Run the pipeline again by clicking on "Start" (top right in the Pipeline view)
  - note, only the new data is ingested
- Select "Full Refresh all" from the "Start" button
  - note that all tables will be recomputed and backfilled

- Could we convert the Materialized View (MV) used for data ingestion to a Streaming Table (ST)
- Use the button "Select Table for Refresh" and select all silver tables to be refreshed only

UC and Lineage

Watch your instructor explaining UC lineage with DLT and the underlying Delta Tables

Delta Tables

(Instructor Demo)

Delta Live Tables is an abstraction for Spark Structured Streaming and built on Delta tables. Delta tables unify DWH, data engineering, streaming and DS/ML.

- Check out Delta table details
  - When viewing the Pipeline Graph select the table "raw_txs"
    - on the right hand side, click on the link under "Metastore" for this table to see table details
    - How many files does that table consist of?
    - Check the generator notebook (/$./00-Loan-Data-Generator) to estimate the number of generated files
- Repeat the same exercise, but start with the navigation bar on the left
  - Click on "Data"
  - Select your catalog / schema. The name of your schema is the **user_id** parameter of your pipeline setting.
  - Drill down to the `raw_tx` table
  - Check the table's schema and sample data

DLT Pipelines in Python (Instructor only)

Listen to your instructor explaining DLT pipelines written in Python. You won't need to run this pipeline.

```
Following the explanations, make sure you can answer the following ques
* Why would you use DLT in Python? (messaging broker[can be done in SQL
* How could you create a DLT in Python?
```

(some hints) (https://docs.databricks.com/workflows/delta-live-tables/delta-live-tables-incremental-data.html)

Direct Publishing Mode

With direct publishing mode you can create pipeline tables under any schema name. Under "Pipeline Settings" the default schema name is set. This setting can be overwritten in the SQL code for every table.

Try using this feature and put the three gold tables into the USER_ID_dashboard schema.

Monitor DLT Events (Optional)

Watch your instructor explaining you how to retrieve DLT events, lineage and runtime data from expectations.

3. Spark with Serverless Compute

You can now use Notebooks to run serverless Spark jobs

- On the left menue bar, click on +New Notebook
- Edit the name of the notebook
- Make sure next to the notebook's name `Python` is displayed for the default cell type (or change it)
- Make sure on the right hand side you see a green dot and `connected`. Click on that button to verify you are connected to `serverless compute` (if not, connect to serverless compute)

Use /explain and /doc

- add the following command to a Python cell, then run it by clicking on the triangle (or using SHIFT-RETURN shortcut):

```
display(spark.range(10).toDF("serverless"))
```

- Click on the symbol for Databricks Assistant and document the the cell. Hint: use /doc in the command line for Assistant and accept the suggestion.

Use /fix

- Add another Python cell and copy the following command into that cell. The command contains a syntax error.
  ```
  display(spark.createDataFrame([("Python",), ("Spark",), ("Databricks
  ```

Click on the Assistant toggle (the blueish/redish star) and try to fix the problem with `/fix` and run the command.

Note that the Assistant is context aware and knows about table names and

3. DWH View / SQL Persona

The Lakehouse unifies classic data lakes and DWHs. This lab will teach you how to access Delta tables generated with a DLT data pipeline from the DWH.

Use the SQL Editor

- On the left menue bar, select the SQL persona
- Also from the left bar, open the SQL editor
- Create a simple query:
  - `SELECT * FROM demo.USER_ID.ref_accounting_treatment` (make sure to use **your schema and table name**)
  - run the query by clicking Shift-RETURN
  - Save the query using your ID as a query name

4. Databricks Workflows with DLT

Create a Workflow

- In the menue bar on the left, select Workflows
- Select "Workflows owned by me"
- Click on "Create Job"
- Name the new job same as **your user_id** from above

Add a first task

- Task name: Ingest
- Task type: DLT task
- Pipeline: your DLT pipeline name for the DLT SQL notebook from above (the pipeline should be in triggered mode for this lab.)

Add a second task

- Task name: Update Downstream
- Task type: Notebook
- Select the `04-Udpate-Downstream` notebook

- Note that `Serverless` is automtatically selected for compute on the right hand side

Run the workflow

- Run the workflow from the "Run now" buttom top right
  - The Workflow will fail with an error in the second task.
  - Switch to the Matrix view.
    - To explore the Matrix View, run the workflow again (it will fail again).

Repair and Rerun (OPTIONAL)

- In the Matrix View, click on the second task marked in red to find out what the error is
  - Click on "Highlight Error"
- Debug the 04-Udpate-Downstream notebook (just comment out the line where the error is caused with `raise`)
- Select the Run with the Run ID again and view the Task
- Use the "Repair and Rerun" Feature to rerun the workflow
  - It should successfully run now.
- You can delete the other failed run

5. Outlook (optional topics in preview)

Follow your instructor for cababilities such as Genie Data Rooms. Time permitting, he will demo them in another environment.

A full end to end demo of this section is available as a video in the Databricks Demo Center (https://www.databricks.com/resources/demos/videos/data-engineering/databricks-data-intelligence-platform?itm_data=demo_center)

Congratulations for completing this workshop!